

Hackathon Information Sheet

DS6051 – Decoding Large Language Models

Date	July 8, 9am - 3pm
Format	In-group, hands-on build session with a final presentation
Group Size	3–5 students per group
Submission	Devpost: project overview, GitHub repo, results table, ~5 min video
Peer Feedback Due	EOD July 10

Overview

Over a single six-hour session, your group will build a project using Gemma LLMs. Each group selects the domain/use-case they will target. All code implementations will be done during the hackathon. The project will focus on safety and involve understanding the limitations of transformer model architecture.

What We Provide

- **The model(s) you will work with:** [gemma-4-E2B](#) and [gemma-4-E2B-it](#) (instruction-tuned). These are small, open-weight models available on Hugging Face. While you can choose to focus on just one, comparing instruction-tuned vs. non-instruction-tuned behavior is encouraged.
- **Boilerplate code for model inference.**
- **Boilerplate code for an LLM-as-judge setup** — a scaffold for using a second model to score open-ended outputs.
- **Access to Rivanna** — 1 GPU (24GB) per group for the duration of the hackathon.
 - account=ds6051-summer
 - partition=interactive
 - reservation=ds6051-summer-hackathon
 - gres=gpu:1
- **Extra GPU access:** the free tiers of Google Colab and Kaggle provide T4 GPUs with 16 GB VRAM. This should be enough to run inference on the model, though likely not enough to also run a judge model simultaneously — plan your judge runs accordingly.

To Do Before the Hackathon

Treat the items below as a checklist — every box should be ticked before July 8, so the session is spent building, not configuring.

- ☐ Pick a domain and a specific use-case (suggestions below).
- ☐ Create a list of datasets relevant to your domain/use-case.
- ☐ Create a Devpost account, join the hackathon (<https://ds6051-hackathon.devpost.com>) and create your group.

- Ensure every group member has access to and some familiarity with the following:
 - Hugging Face (including an API key – save this somewhere!)
 - GitHub
- Ensure at minimum one group member has access to and high familiarity with the following:
 - Rivanna
 - LLMs as judges (we will share this!)
- Optional but useful (to avoid headaches and time lost day-of):
 - Setup your environment. Ensure each team member who will be coding (across local compute, Colab, Kaggle, Rivanna) is using the same version of python and common packages (e.g. torch). Use virtual environments and requirements.txt files for ease.

Potential Domains / Use-Cases

The list below is a set of starting points. You are welcome to propose another domain or a different use-case — the more concrete your use-case, the sharper your safety choices can be.

- 1. Legal**
 - Document review assistant
 - Legal advice chatbot
- 2. Medical / Health**
 - Clinical decision-support for clinicians
 - Patient-facing chatbot (could be a symptom checker or insurance navigator)
 - Mental-health support companion or therapist
- 3. Education**
 - Subject tutor/ Educational chatbot (kindergarten/elementary level)
 - Subject tutor (high school/college/graduate level)
 - Automated grader / essay scorer
 - Teacher's lesson-plan assistant
- 4. Cybersecurity**
 - Alert triage / SOC analyst assistant
 - Phishing-email detector
 - Secure-code reviewer
 - Content moderation
- 5. Business / Enterprise**
 - Internal knowledge-base assistant
 - Customer-support chatbot
 - HR / hiring screener
 - Financial advice / planning assistant
 - Analytics assistant
 - Code generation for app development
 - Insider trading
- 6. Biosecurity**
- 7. Climate**

Final Product

Each group submits the following on Devpost:

- **Code on GitHub**
- **A table of your results**
- **Discussion**
- **Devpost submission:** upload a project overview, GitHub repo, results table, written discussion, and ~5-minute video

Grading

Hackathon Submission

Your project, code, discussion, and video are assessed together. Strong submissions show thoughtful choices, working implementations, and — importantly — **honest reflection on the limitations given current LLM architecture**.

The Hackathon project and will be a total of 80 points as broken down below:

1. Innovation and Creativity (20 pts): Is this a new idea or unique approach to an old problem?
2. Technical Implementation (30 pts): Does the prototype work?
3. Problem-Solving & Impact (20 pts): Does this project solve a real, meaningful problem in AI safety? Are the choices and limitations clearly discussed?
4. Presentation/Pitch (10 pts): Presentation of your results on Devpost!

Discussion Grade — Peer Feedback

As part of your discussion grade, view and provide comments on **2 other groups'** hackathon submissions via Devpost. **Due by EOD July 10.**

For each comment, please include each of the following four responses:

Pluses, Potential, Concerns, Overcoming Concerns (PPCO)

PPCO is a four-step technique for evaluating and improving an idea. Rooted in affirmative judgment, PPCO lets you express both positive and negative responses to an idea or to provide feedback on something that has occurred, all without crushing its potential OR originator!

